



Home Work

DevOps

HW_L3_01_Docke

r

۱. فایل تمرین را در پنل خود آپلود کنید.

۲. title فایل تمرین به صورت (نام تمرین+نام و نام خانوادگی) به انگلیسی باشد.

نمونه: AmirMohammadMohammadi_docker_01

۳. در صورتی که سوال و یا ابهامی دارید در گروه چت تلگرامی بپرسید.

تمرین Docker

مجموع امتیاز: هشتاد نمره

- تمام سناریوها روی یک سیستم Ubuntu 22.04 یا WSL/VM لوکال قابل اجرا هستند
- برای هر سوال فایل و کانفیگ نمونه داده شده تا دقیقاً همان را بسازید
- تحویل یک پوشه کامل شامل پوشه‌های هر سناریو، کانفیگ‌ها و خروجی دستورها

ساختار پیشنهادی پوشه کار:

docker_homework/

|— 01_docker_images/

|— 02_containers_1/

|— 03_containers_2/

|— 04_volumes_network_1/

|— 05_volumes_network_2/

|— 06_docker_compose/

|— 07_sample_project/

مجموعه سوالات اول Image: و Docker — یازده نمره

سوال ۱.۱ — نصب و بررسی نسخه Docker — چهار نمره

آنچه می‌خواهیم: نصب Docker و بررسی وضعیت آن

• فایل اسکریپت ایجاد کنید 01_docker_images/install_docker.sh :

• نمونه محتوای اسکریپت جاهای TODO را تکمیل کنید:

```
#!/bin/bash
```

```
set -e
```

```
# Docker نصب (برای Ubuntu/Debian)
```

```
sudo apt update
```

```
sudo apt install -y docker.io docker-compose
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

بررسی نسخه

```
docker --version > docker_version.txt
```

```
docker info | head -n 20 >> docker_version.txt
```

- خروجی دستورهای زیر را در 01_docker_images/docker_status.txt ذخیره کنید:

```
sudo systemctl status docker --no-pager | head -n 15
```

```
docker ps -a
```

Hints:

اگر از WSL استفاده می‌کنید، ممکن است نیاز به تنظیمات اضافی باشد؛ در فایل توضیح دهید

برای اجرای دستورات Docker بدون sudo ، کاربر خود را به گروه docker اضافه کنید sudo :
usermod -aG docker \$USER

سوال ۱.۲ — کار با Images شامل Pull ، Tag و Inspect — چهار نمره

- دستورهای زیر را اجرا و خروجی‌ها را

در 01_docker_images/image_operations.txt ذخیره کنید:

```
docker pull nginx:alpine
```

```
docker images | grep nginx
```

```
docker tag nginx:alpine my-nginx:v1.0
```

```
docker images | grep -E 'nginx|my-nginx'
```

```
docker inspect nginx:alpine | grep -A 5 "Architecture"
```

```
docker rmi my-nginx:v1.0
```

- یک فایل 01_docker_images/image_summary.txt بسازید و به این سوالات پاسخ دهید:

۱. حجم image nginx:alpine چقدر است؟

۲. Architecture این image چیست؟

۳. تفاوت docker pull و docker run چیست؟

Hints:

از `docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"` نمایش مرتب‌تر استفاده کنید

سوال ۱.۳ — بررسی Layers و History در Images سه نمره

- دستورهای زیر را در 01_docker_images/image_layers.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
docker history nginx:alpine --no-trunc > image_history.txt
```

```
docker inspect nginx:alpine | grep -A 10 "RootFS" > image_rootfs.txt
```

```
docker image inspect nginx:alpine --format '{{.Size}}' > image_size_bytes.txt
```

- یک فایل 01_docker_images/layers_explanation.txt بسازید و به این سوالات پاسخ دهید:

۱. چند layer در image nginx:alpine وجود دارد؟

۲. بزرگ‌ترین layer کدام است؟

۳. چرا استفاده از image های کوچک مثل alpine بهتر است؟

Hints:

از docker history برای مشاهده لایه‌های image استفاده کنید

هر دستور در Dockerfile یک layer جدید ایجاد می‌کند

مجموعه سوالات دوم - Containers: بخش اول — دوازده نمره

سوال ۲.۱ — اجرای Container ساده و بررسی وضعیت آن — سه نمره

- دستورهای زیر را در 02_containers_1/container_basics.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
# ساده container اجرای یک
```

```
docker run -d --name test-nginx nginx:alpine
```

```
sleep 2
```

```
docker ps | grep test-nginx
```

```
docker logs test-nginx | head -n 10 > nginx_logs.txt
```

```
docker stop test-nginx
```

```
docker ps -a | grep test-nginx
```

- خروجی دستور `docker inspect test-nginx | grep -A 3 "State"` در `02_containers_1/container_state.txt` ذخیره کنید

Hints:

اگر container قبلاً وجود داشت، ابتدا آن را حذف کنید `docker rm -f test-nginx` :

سوال ۲.۲ — کار با متغیرهای Environment سه نمره

- یک container با متغیر محیطی اجرا کنید:

```
docker run -d --name env-test -e MY_VAR="Hello Docker" -e PORT=8080  
alpine:latest sleep 3600
```

- دستورهای زیر را در `02_containers_1/env_check.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
docker exec env-test env | grep -E 'MY_VAR|PORT' > env_output.txt
```

```
docker exec env-test sh -c 'echo "MY_VAR=$MY_VAR, PORT=$PORT"'  
>> env_output.txt
```

- خروجی فایل env_output.txt را در پوشه تحویل قرار دهید

Hints:

از docker exec برای اجرای دستورات داخل container استفاده کنید

سوال ۲.۳ Port Mapping — و دسترسی به Container سه نمره

- یک container با port mapping اجرا کنید:

```
docker run -d --name web-server -p 8888:80 nginx:alpine
```

- دستورهای زیر را در 02_containers_1/port_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 2
```

```
curl -I http://localhost:8888 > port_mapping_test.txt
```

```
docker port web-server >> port_mapping_test.txt
```

```
ss -tuln | grep 8888 >> port_mapping_test.txt
```

- خروجی فایل port_mapping_test.txt را در پوشه تحویل قرار دهید

Hints:

اگر پورت ۸۸۸۸ اشغال بود، از پورت دیگری استفاده کنید و در فایل توضیح دهید

سوال ۲.۴ — کار با docker exec و حالت Interactive سه نمره

- یک container اجرا کنید:

```
docker run -d --name exec-test alpine:latest sleep 3600
```

- دستورهای زیر را در 02_containers_1/exec_operations.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
docker exec exec-test sh -c 'echo "Hello from exec" > /tmp/test.txt'
```

```
docker exec exec-test cat /tmp/test.txt > exec_output.txt
```

```
docker exec exec-test sh -c 'ps aux' > exec_processes.txt
```

```
docker exec -it exec-test sh -c 'echo "Interactive exec test" >> /tmp/test.txt'  
2>&1 | head -n 5 >> exec_output.txt
```

- یک فایل 02_containers_1/exec_explanation.txt بسازید و تفاوت docker

exec و docker attach را توضیح دهید

Hints:

از docker exec برای اجرای دستورات در container در حال اجرا استفاده کنید

docker attach به process اصلی container متصل می‌شود

مجموعه سوالات سوم - Containers: بخش دوم — دوازده نمره

سوال ۳.۱ — کار با Logs در Container سه نمره

- یک container با لاگ‌های متعدد اجرا کنید:

```
docker run -d --name log-test alpine:latest sh -c 'for i in $(seq 1 10); do echo "Log line $i"; sleep 1; done; sleep 3600'
```

- دستورهای زیر را در 03_containers_2/log_operations.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 12
```

```
docker logs log-test > all_logs.txt
```

```
docker logs --tail 5 log-test > last_5_logs.txt
```

```
docker logs --since 5s log-test > recent_logs.txt
```

- فایل‌های خروجی را در پوشه تحویل قرار دهید

Hints:

از `docker logs -f` برای دنبال کردن لاگ‌ها به صورت real-time استفاده کنید اما در اسکریپت نباشد

سوال ۳.۲ Restart Policy — و — Auto-restart سه نمره

- سه container با restart policy های مختلف اجرا کنید:

```
docker run -d --name always-restart --restart always alpine:latest sleep 5
```

```
docker run -d --name unless-stopped --restart unless-stopped alpine:latest sleep 5
```

```
docker run -d --name no-restart alpine:latest sleep 5
```

- دستورهای زیر را در 03_containers_2/restart_policy_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 7
```

```
docker ps -a --format "table {{.Names}}\t{{.Status}}\t{{.RestartCount}}" |  
grep -E 'always-restart|unless-stopped|no-restart' > restart_status.txt
```

```
docker inspect always-restart | grep -A 2 "RestartPolicy" >> restart_status.txt
```

- خروجی را در پوشه تحویل قرار دهید

Hints:

Containerهایی که 5 sleep دارند، بعد از ۵ ثانیه متوقف می‌شوند؛ restart policy تعیین می‌کند که آیا دوباره شروع شوند

سوال ۳.۳ — محدودیت‌های Resource سه نمره

- یک container با محدودیت حافظه و CPU اجرا کنید:

```
docker run -d --name limited-container --memory="128m" --cpus="0.5"  
alpine:latest sleep 3600
```

- دستورهای زیر را در 03_containers_2/resource_check.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
docker stats --no-stream limited-container > resource_stats.txt
```

```
docker inspect limited-container | grep -A 5 "Memory\|CpuShares" >>
resource_stats.txt
```

- خروجی را در پوشه تحویل قرار دهید

Hints:

از `docker stats` برای مشاهده استفاده واقعی منابع استفاده کنید

سوال ۳.۴ — بررسی — Health Checks سه نمره

- یک container با health check اجرا کنید:

```
docker run -d --name health-test --health-cmd="curl -f http://localhost:80 | |
exit 1" --health-interval=5s --health-timeout=3s --health-retries=3 nginx:alpine
```

- دستورهای زیر را در `03_containers_2/health_check.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 10
```

```
docker inspect --format='{{.State.Health.Status}}' health-test >
health_status.txt
```

```
docker inspect --format='{{json .State.Health}}' health-test | python3 -m
json.tool > health_details.txt
```

```
docker ps --filter "name=health-test" --format "table
{{.Names}}\t{{.Status}}" >> health_status.txt
```

- یک فایل `03_containers_2/health_explanation.txt` بسازید و به این سوالات پاسخ دهید:

۱. وضعیت health check چیست؟

۲. چه زمانی health check اجرا می‌شود؟

۳. چرا health checks مهم هستند؟

Hints:

Health checks به Docker کمک می‌کنند تا وضعیت container را بررسی کند

از `docker inspect` برای مشاهده جزئیات health check استفاده کنید

مجموعه سوالات چهارم Volumes: Network - بخش اول — یازده نمره

سوال ۴.۱ — ایجاد و استفاده از Volume با نام — چهار نمره

- یک named volume ایجاد و استفاده کنید:

```
docker volume create my-data
```

```
docker run -d --name volume-test -v my-data:/app/data alpine:latest sleep 3600
```

- دستورهای زیر را در `04_volumes_network_1/volume_operations.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
docker exec volume-test sh -c 'echo "Hello from container" > /app/data/test.txt'
```

```
docker exec volume-test cat /app/data/test.txt > volume_content.txt
```

```
docker volume inspect my-data > volume_info.txt
```

```
docker rm -f volume-test
```

```
docker run -d --name volume-test-2 -v my-data:/app/data alpine:latest sleep 3600
```

```
docker exec volume-test-2 cat /app/data/test.txt >> volume_content.txt
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

Named volumes داده‌ها را حتی بعد از حذف container حفظ می‌کنند

سوال ۴.۲ — استفاده از Bind Mount برای توسعه — چهار نمره

- یک فایل HTML ساده در 04_volumes_network_1/web/index.html بسازید:

```
<html>
```

```
<body>
```

```
<h1>Docker Bind Mount Test</h1>
```

```
<p>This file is mounted from host.</p>
```

```
</body>
```

```
</html>
```

- یک container با bind mount اجرا کنید:

```
docker run -d --name bind-mount-test -p 9000:80 -v  
$(pwd)/04_volumes_network_1/web:/usr/share/nginx/html:ro nginx:alpine
```

- دستورهای زیر را در 04_volumes_network_1/bind_mount_test.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 2
```

```
curl http://localhost:9000 > bind_mount_output.html
```

```
echo "File modified on host" >> 04_volumes_network_1/web/index.html
```

```
curl http://localhost:9000 > bind_mount_output_modified.html
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

با bind mount، تغییرات در host بلافاصله در container قابل مشاهده است

سوال ۴.۳ Backup — و Restore در Volume سه نمره

- یک volume با داده ایجاد کنید:

```
docker volume create backup-test
```

```
docker run --rm -v backup-test:/data alpine:latest sh -c 'echo "Important data"  
> /data/file.txt && echo "More data" > /data/file2.txt'
```

- دستورهای زیر را در 04_volumes_network_1/volume_backup.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
# Backup volume
```

```
docker run --rm -v backup-test:/data -v $(pwd):/backup alpine:latest tar czf  
/backup/volume_backup.tar.gz -C /data .
```

```
ls -lh volume_backup.tar.gz > backup_info.txt
```

```
# Restore to new volume
```

```
docker volume create restore-test
```

```
docker run --rm -v restore-test:/data -v $(pwd):/backup alpine:latest tar xzf  
/backup/volume_backup.tar.gz -C /data
```

```
docker run --rm -v restore-test:/data alpine:latest cat /data/file.txt >  
restored_data.txt
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

از tar برای backup و restore volume استفاده کنید

همیشه قبل از حذف volume ، backup بگیرید

مجموعه سوالات پنجم Volumes و: Network بخش دوم — یازده نمره

سوال ۵.۱ — ایجاد Network سفارشی — چهار نمره

- یک custom network ایجاد و دو container را به آن متصل کنید:

```
docker network create my-network
```

```
docker run -d --name container1 --network my-network alpine:latest sleep 3600
```

```
docker run -d --name container2 --network my-network alpine:latest sleep 3600
```

- دستورهای زیر را در 05_volumes_network_2/network_operations.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
docker network inspect my-network > network_info.txt
```

```
docker exec container1 ping -c 3 container2 > ping_test.txt
```

```
docker exec container1 nslookup container2 >> ping_test.txt
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

در یک custom network ، container ها می‌توانند با نام container به یکدیگر دسترسی داشته باشند

سوال ۵.۲ — ارتباط بین Container ها — چهار نمره

- یک container با database simulation و یک container client اجرا کنید:

```
docker run -d --name db --network my-network -e
```

```
MYSQL_ROOT_PASSWORD=test123 mysql:8.0
```

```
docker run -d --name client --network my-network alpine:latest sleep 3600
```

- دستورهای زیر را در 05_volumes_network_2/container_communication.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
sleep 10
```

```
docker exec client ping -c 2 db > db_connectivity.txt
```

```
docker exec client nslookup db >> db_connectivity.txt
```

```
docker network ls > all_networks.txt
```

```
docker network inspect bridge | grep -A 10 "Containers" >> all_networks.txt
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

Containerها در یک network می‌توانند با نام container به یکدیگر متصل شوند

سوال ۵.۳ Isolation — در Network و — Bridge Network سه نمره

- دو network جداگانه ایجاد کنید:

```
docker network create network-a
```

```
docker network create network-b
```

```
docker run -d --name container-a --network network-a alpine:latest sleep 3600
```

```
docker run -d --name container-b --network network-b alpine:latest sleep 3600
```

- دستورهای زیر را در 05_volumes_network_2/network_isolation.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
docker exec container-a ping -c 2 container-b 2>&1 > isolation_test.txt ||  
echo "Ping failed as expected" >> isolation_test.txt
```

```
docker network connect network-a container-b
```

```
docker exec container-a ping -c 2 container-b >> isolation_test.txt
```

```
docker network inspect network-a | grep -A 5 "Containers" >  
network_a_containers.txt
```

```
docker network inspect bridge | grep -A 10 "IPAM" >  
bridge_network_info.txt
```

- یک فایل 05_volumes_network_2/isolation_explanation.txt بسازید و به این سوالات پاسخ دهید:

۱. چرا container-a نمی‌توانست به container-b ping بزند؟

۲. بعد از docker network connect چه اتفاقی افتاد؟

۳. تفاوت bridge network و custom network چیست؟

Hints:

Network های جداگانه container ها را از یکدیگر ایزوله می‌کنند

یک container می‌تواند به چند network متصل باشد

مجموعه سوالات ششم — Docker Compose: سیزده نمره

فایل docker-compose.yml

- فایل زیر را در 06_docker_compose/docker-compose.yml ذخیره کنید:

```
version: '3.8'
```

```
services:
```

```
  web:
```

```
    image: nginx:alpine
```

```
    ports:
```

```
      - "8080:80"
```

```
    volumes:
```

```
      - ./web:/usr/share/nginx/html:ro
```

```
    depends_on:
```

```
      - app
```

app:

image: alpine:latest

command: sh -c "apk add --no-cache python3 && python3 -m http.server 5000"

ports:

- "5000:5000"

volumes:

- ./app:/app

working_dir: /app

db:

image: mysql:8.0

environment:

MYSQL_ROOT_PASSWORD: rootpass

MYSQL_DATABASE: testdb

volumes:

- db-data:/var/lib/mysql

volumes:

db-data:

سوال ۶.۱ — راه‌اندازی Compose و بررسی سرویس‌ها — پنج نمره

- پوشه‌های لازم را ایجاد کنید:

```
mkdir -p 06_docker_compose/web 06_docker_compose/app
```

```
echo "<h1>Web Service</h1>" > 06_docker_compose/web/index.html
```

```
echo "print('App Service')" > 06_docker_compose/app/app.py
```

- دستورهای زیر را در 06_docker_compose/compose_operations.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
cd 06_docker_compose
```

```
docker-compose up -d
```

```
sleep 5
```

```
docker-compose ps > compose_status.txt
```

```
docker-compose logs web | head -n 10 > web_logs.txt
```

```
curl http://localhost:8080 > web_response.html
```

```
docker-compose down
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

از `docker-compose up -d` برای اجرای سرویس‌ها در `background` استفاده کنید

سوال ۶.۲ — بررسی Network و Volume در Compose — پنج نمره

- دستورهای زیر را در `06_docker_compose/compose_inspect.sh` ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
cd 06_docker_compose
```

```
docker-compose up -d
```

```
sleep 5
```

```
docker-compose exec web ping -c 2 app > service_communication.txt
```

```
docker network ls | grep docker_compose > compose_networks.txt
```

```
docker volume ls | grep docker_compose > compose_volumes.txt
```

```
docker-compose down -v
```

- یک فایل `06_docker_compose/compose_summary.txt` بسازید و به این سوالات پاسخ

دهید:

۱. چه `network` هایی توسط `docker-compose` ایجاد شد؟

۲. چه `volume` هایی ایجاد شد؟

۳. چگونه سرویس `web` به سرویس `app` دسترسی دارد؟

Hints:

از `docker-compose config` برای مشاهده کانفیگ نهایی استفاده کنید

سوال ۶.۳ — متغیرهای Environment و فایل `.env` سه نمره

- یک فایل `.env` در `06_docker_compose/.env` بسازید:

```
MYSQL_ROOT_PASSWORD=mysecretpassword
```

```
MYSQL_DATABASE=mydb
```

```
APP_PORT=3000
```

- فایل `docker-compose.yml` را به‌روزرسانی کنید تا از `environment variables` استفاده کند:

```
version: '3.8'
```

```
services:
```

```
  web:
```

```
    image: nginx:alpine
```

```
    ports:
```

```
      - "${APP_PORT:-8080}:80"
```

```
    environment:
```

- APP_NAME=MyApp

volumes:

- ./web:/usr/share/nginx/html:ro

db:

image: mysql:8.0

environment:

MYSQL_ROOT_PASSWORD: \${MYSQL_ROOT_PASSWORD}

MYSQL_DATABASE: \${MYSQL_DATABASE}

volumes:

- db-data:/var/lib/mysql

volumes:

db-data:

ذخیره و اجرا کنید 06_docker_compose/env_test.sh دستورهای زیر را در

```
#!/bin/bash
```

```
cd 06_docker_compose
```

```
docker-compose config > compose_with_env.txt
```

```
docker-compose up -d
```

```
sleep 5
```

```
docker-compose exec db env | grep MYSQL > db_env_vars.txt
```

```
docker-compose down
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

از `env` file برای مدیریت `environment variables` استفاده کنید

`docker-compose config` کانفیگ نهایی با جایگزینی متغیرها را نشان می‌دهد

مجموعه سوالات هفتم — Sample Project: هشت نمره

پروژه: یک وب‌اپلیکیشن ساده با `Python` و `Nginx`

- ساختار پروژه را ایجاد کنید:

```
07_sample_project/
```

```
├── docker-compose.yml
```

```
├── nginx/
```

```
│   └── nginx.conf
```

```
└── app/
```

```
| └─ app.py
└─ web/
    └─ index.html
```

فایل‌های پروژه

07_sample_project/docker-compose.yml:

```
version: '3.8'
```

```
services:
```

```
  nginx:
```

```
    image: nginx:alpine
```

```
    ports:
```

```
      - "3000:80"
```

```
    volumes:
```

```
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
```

```
      - ./web:/usr/share/nginx/html:ro
```

```
    depends_on:
```

```
      - python-app
```

python-app:

build: ./app

volumes:

- ./app:/app

working_dir: /app

networks:

default:

name: sample-project-network

07_sample_project/nginx/nginx.conf:

events {}

http {

server {

listen 80;

location / {

root /usr/share/nginx/html;

index index.html;

}

```

location /api {

    proxy_pass http://python-app:8000;

}

}

}

07_sample_project/app/app.py:

from http.server import BaseHTTPRequestHandler, HTTPServer

class Handler(BaseHTTPRequestHandler):

    def do_GET(self):

        self.send_response(200)

        self.send_header('Content-type', 'application/json')

        self.end_headers()

        self.wfile.write(b'{"message": "Hello from Python API"}')

if __name__ == '__main__':

    server = HTTPServer(('0.0.0.0', 8000), Handler)

    server.serve_forever()

```

07_sample_project/app/Dockerfile:

FROM python:3.9-alpine

WORKDIR /app

COPY app.py .

CMD ["python", "app.py"]

07_sample_project/web/index.html:

<html>

<head><title>Sample Project</title></head>

<body>

<h1>Docker Sample Project</h1>

<p>Frontend served by Nginx</p>

<button

onclick="fetch('/api').then(r=>r.json()).then(d=>alert(d.message))">Call
API</button>

</body>

</html>

سوال ۷.۱ — راه اندازی و تست پروژه — چهار نمره

- دستورهای زیر را در 07_sample_project/run_project.sh ذخیره و اجرا کنید:

```
#!/bin/bash
```

```
cd 07_sample_project
```

```
docker-compose up -d --build
```

```
sleep 5
```

```
curl http://localhost:3000 > frontend_test.html
```

```
curl http://localhost:3000/api > api_test.json
```

```
docker-compose ps > project_status.txt
```

```
docker-compose logs --tail 20 > project_logs.txt
```

- خروجی فایل‌ها را در پوشه تحویل قرار دهید

Hints:

از `docker-compose up --build` برای `build` کردن `image` های سفارشی استفاده کنید

سوال ۷.۲ — مستندسازی پروژه — چهار نمره

- یک فایل `07_sample_project/README.md` بسازید و شامل موارد زیر باشد:

۱. توضیح کوتاه پروژه

۲. نحوه راه‌اندازی دستورات `docker-compose`

۳. ساختار فایل‌ها و نقش هر سرویس

۴. نحوه دسترسی به سرویس‌ها پورت‌ها و URL ها

- یک فایل 07_sample_project/architecture.txt بسازید و به این سوالات پاسخ دهید:

۱. چگونه Nginx به Python app متصل می‌شود؟

۲. چه network هایی در این پروژه استفاده شده؟

۳. اگر بخواهید یک database اضافه کنید، چه تغییراتی باید بدهید؟

Hints:

- از Markdown برای خوانایی بهتر استفاده کنید

تحويل نهایی و فایل خروجی — دو نمره

- یک فایل README.md در ریشه پوشه تمرین قرار دهید و ساختار پوشه خود را خلاصه کنید
- خروجی دستور tree یا لیست find را در فایل final_structure.txt ذخیره کنید تا مشخص باشد چه فایل‌هایی تحويل داده‌اید
- یک فایل commands_history.txt شامل تمام دستورات Docker که اجرا کرده‌اید می‌توانید از `history | grep docker` استفاده کنید

معیار نمره‌دهی خلاصه

- اجرای صحیح container ها و سرویس‌ها: سی درصد

- کار با volumes و networks: بیست درصد
 - استفاده صحیح از docker-compose: بیست درصد
 - رعایت دقیق مسیرها و نام فایل‌ها: ده درصد
 - توضیحات کوتاه و مرتب در فایل‌های متنی: ده درصد
 - سازمان‌دهی پوشه و تحویل تمیز: ده درصد
- مهلت تحویل: چهارشنبه هفته آینده